PRODUCT AND ENGINEERING AUDIT

StyledGenie Complete Implementation Roadmap

Combined plan for demo readiness, secure single-store production, and a public multi-merchant Shopify SaaS launch.

Audit date	10 August 2026
Scope	72 implementation items across 8 workstreams
Codebase	Storefront widget, merchant dashboard, FastAPI backend, Shopify extension, Supabase
Planning range	8-12 weeks for one developer to reach public SaaS readiness

Prepared from a read-only audit of the running application, source code, configuration, data readiness, and existing project documentation.

1. Executive summary

StyledGenie already has meaningful styling and support logic. The correct next move is to stabilize and secure it, not rebuild it.

Bottom line: 72 scoped implementation items. Demo-ready requires about 7-12 developer-days. Secure single-store production requires about 24-36 developer-days. Public multi-merchant SaaS readiness requires about 40-61 developer-days including contingency, or roughly 8-12 weeks for one developer.

What is already working

- FastAPI backend serves a live storefront demo and merchant dashboard.
- OpenAI, LangChain orchestration, and Google Vision report ready in the running workspace.
- Find My Outfit includes profile extraction, decision mode, recommendations, explanations, and refinement.
- Complete My Look contains anchor analysis, gap analysis, palette logic, and complementary selection.
- Get Inspired contains hero-first ranking and supporting-item selection.
- Smart Swap replaces one requested category while retaining outfit context.
- Support contains routing, tracking, return/exchange, sizing, issue-image, and handoff logic.
- Merchant APIs cover catalog, workspace, customization, knowledge, care setup, description generation, looks, and analytics.

What prevents production release

- The local storefront and Shopify-delivered widget are materially different versions.
- The main shopify.app.toml file is not valid TOML; it contains a Python patch script.
- Merchant administration endpoints do not enforce Shopify authentication.
- Data access is centered on one DEFAULT_MERCHANT_ID rather than authenticated shop tenancy.
- There is no automated test suite for the mandatory product rules.
- Support logic is not connected to real operational content: zero synced orders, zero configured FAQs, and no notification provider.
- Commerce attribution is present conceptually but no assisted orders or revenue are recorded.

2. Evidence-backed audit snapshot

Area	Observed state	Assessment
Runtime	Backend health and all inspected GET surfaces returned HTTP 200.	Working
Catalog	840 products; all reported with images and links.	Strong base
Catalog intelligence	152 tagged products; 796 tags; average 5.95 tags on tagged/available data.	Incomplete coverage
Catalog freshness	Last recorded sync was about 167.7 hours old at audit time.	Needs automation
AI stack	OpenAI, LangChain tools, and Google Vision REST reported ready.	Working, untested
Chat usage	545 interactions across 36 sessions; 66 outfit and 26 image recommendation events.	Active data
Complete My Look	No completed journey events recorded in analytics.	Needs E2E proof
Support content	0 merchant FAQs, 1 knowledge entry, 0% support coverage; endpoint uses fallback FAQs.	Not operational
Orders	Orders scope ready, but 0 synced orders and 0 assisted orders.	Not validated
Curated looks	0 saved looks.	Missing merchant content
Testing	Syntax checks pass; only a manual QA checklist is present.	High regression risk

Repository-specific structural findings

Priority	Finding	Impact
P0	Widget drift	apps/storefront-widget/app.js is much newer and larger than backend/static and the Shopify extension asset. The demo and deployed extension are not one release artifact.
P0	Invalid deploy config	shopify.app.toml starts with Python code instead of Shopify TOML configuration.
P0	Missing admin boundary	Merchant routes perform reads and writes without a Depends/authentication guard.
P0	Single-store resolution	SupabaseService repeatedly resolves DEFAULT_MERCHANT_ID rather than authenticated shop context.
P0	Open CORS	The backend enables wildcard origins with credentials.
P1	Large modules	The conversation and recommendation services are each several thousand lines, increasing regression and review cost.
P1	Synchronous external work	Catalog, Vision, OpenAI, Shopify, and notification calls can occupy request workers and need controlled retries/jobs.
P1	Data isolation	The checked schema has merchant_id columns but no visible row-level security policies.

3. Total scope, effort, and release choices

Effort ranges are planning estimates, not delivery guarantees. They assume one developer familiar with the codebase and timely access to Shopify, Supabase, and production credentials.

Workstream	Priority	Items	Estimated effort
W1 - Release stabilization	P0	7	2-3 days
W2 - Core product intelligence and UX	P0	12	4-6 days
W3 - Shopify security and multi-tenancy	P0	11	7-10 days
W4 - Operational customer support	P1	9	4-6 days
W5 - Catalog intelligence and merchant controls	P1	8	4-6 days
W6 - Commerce attribution and analytics	P1	7	3-5 days
W7 - Platform engineering and AI operations	P1	10	6-9 days
W8 - Privacy, commercial readiness, and launch	P2	8	5-8 days
TOTAL		**72**	**35-53 developer-days before contingency**

Three valid finish lines

Target	Included scope	Estimate	Use case
Demo-ready	W1 + core W2 checks + basic support data	7-12 days	Merchant demonstration and controlled development store
Secure single-store production	W1-W7, scoped to one merchant and no public billing	24-36 days	One known merchant on stable hosting
Public Shopify SaaS	All workstreams, contingency, public lifecycle and billing	40-61 days / 8-12 weeks	Multiple merchants and public distribution

Recommended commitment: stabilize the current release first, then target secure single-store production. Treat public SaaS work as a separate approved release because it introduces billing, multi-tenancy, privacy, and operational obligations.

Priority meaning

- P0 - release blocker: deploy integrity, mandatory product behavior, authentication, and tenant isolation.
- P1 - operational requirement: real support, catalog coverage, analytics, resilience, and observability.
- P2 - public SaaS requirement: billing, privacy operations, self-service onboarding, and launch governance.

4. W1 - Release stabilization

Priority	Items	Effort	Objective
P0	7	2-3 days	Make the current codebase deployable and ensure the local demo matches the Shopify storefront.

Implementation details

- W1.1 Review and preserve the current uncommitted changes before additional feature work.
- W1.2 Select apps/storefront-widget as the canonical widget source.
- W1.3 Create a repeatable asset build/sync process for backend static files and the Shopify theme extension.
- W1.4 Restore shopify.app.toml as valid Shopify configuration.
- W1.5 Remove or quarantine captured development artifacts and duplicated obsolete assets.
- W1.6 Add safe startup configuration validation and a release smoke-check command.
- W1.7 Update README, tickets, and run/deploy instructions to match the real architecture.

Acceptance criteria

- Local and Shopify widgets have matching behavior and visual assets.
- The Shopify CLI parses the app configuration and can package the extension.
- A clean checkout can start the backend and render both UI surfaces.

5. W2 - Core product intelligence and UX

Priority	Items	Effort	Objective
P0	12	4-6 days	Prove every mandatory StyledGenie behavior with deterministic automated checks.

Implementation details

- W2.1 Test strict styling/support mode separation in backend responses and frontend rendering.
- W2.2 Test image-first ordering so Vision runs before generic questions.
- W2.3 Test Find My Outfit extraction for event, budget, vibe, weather, and urgency.
- W2.4 Test the mandatory options-versus-best-one decision prompt.
- W2.5 Guarantee decision mode returns one outfit only.
- W2.6 Test Complete My Look anchor identification and missing-category gap analysis.
- W2.7 Test palette, silhouette, occasion, and weather compatibility rules.
- W2.8 Test Get Inspired hero-first ordering with one closest match.
- W2.9 Test Smart Swap changes only the requested category.
- W2.10 Test menswear/womenswear guardrails and ambiguous-segment clarification.
- W2.11 Require concise Why this works explanations on every recommendation.
- W2.12 Add accessibility, mobile, camera, voice, upload, and failure-state QA.

Acceptance criteria

- All mandatory AGENTS.md rules have positive and negative automated tests.
- Support responses cannot render styling cards, chips, or outfit actions.
- External AI failure produces short actionable fallbacks without breaking the flow.

6. W3 - Shopify security and multi-tenancy

Priority	Items	Effort	Objective
P0	11	7-10 days	Turn the current single-store backend into a secure Shopify application boundary.

Implementation details

- W3.1 Implement Shopify OAuth/install lifecycle and embedded-app session-token verification.
- W3.2 Resolve merchant context from the authenticated shop, not DEFAULT_MERCHANT_ID.
- W3.3 Protect all merchant read and write endpoints.
- W3.4 Separate public storefront endpoints from merchant administration endpoints.
- W3.5 Validate signed shop context and customer identifiers from storefront traffic.
- W3.6 Replace wildcard CORS with explicit configured origins.
- W3.7 Store and rotate Shopify credentials per merchant.
- W3.8 Add uninstall, customer-data request, and customer-data erasure handlers.
- W3.9 Enable tenant isolation using Supabase RLS and server-side enforcement.
- W3.10 Add rate limits, request IDs, and audit logs for sensitive writes.
- W3.11 Add merchant and staff role authorization where dashboard collaboration is needed.

Acceptance criteria

- Unauthenticated merchant calls return 401 or 403.
- One shop cannot read or mutate another shop's products, settings, orders, or analytics.
- Install, reinstall, credential refresh, and uninstall paths are tested.

7. W4 - Operational customer support

Priority	Items	Effort	Objective
P1	9	4-6 days	Convert implemented support logic into a real, merchant-configured service.

Implementation details

- W4.1 Sync and validate real development-store orders, fulfillment, returns, and line items.
- W4.2 Require both order number and email before disclosing order information.
- W4.3 Implement explicit live-versus-snapshot status labels.
- W4.4 Configure merchant returns, exchanges, shipping, sizing, and escalation policies.
- W4.5 Complete refund-versus-exchange selection and affected-item identification.
- W4.6 Validate return windows, item eligibility, availability, and replacement sizes.
- W4.7 Configure email handoff first and optional WhatsApp/Twilio second.
- W4.8 Persist notification attempts, retries, ownership, status, and resolution notes.
- W4.9 Keep damaged/wrong-item image handling isolated in support mode.

Acceptance criteria

- A valid order returns real status; an invalid lookup leaks no customer data.
- Returns and exchanges identify the exact order and item before action.
- A handoff is stored, delivered, visible to the merchant, and auditable.

8. W5 - Catalog intelligence and merchant controls

Priority	Items	Effort	Objective
P1	8	4-6 days	Make recommendations consistently usable across the full live catalog.

Implementation details

- W5.1 Add webhook-driven or scheduled product, inventory, variant, and price synchronization.
- W5.2 Auto-tag untagged catalog products in controlled, resumable batches.
- W5.3 Add merchant review and approval before Shopify tag/metafield write-back.
- W5.4 Create and save initial curated looks using real catalog products.
- W5.5 Validate generated product descriptions before Shopify application.
- W5.6 Enforce inventory, market, currency, size, and variant availability.
- W5.7 Make merchant AI behavior and knowledge settings affect shopper responses.
- W5.8 Add data-quality alerts for stale data, missing links, images, variants, and weak categories.

Acceptance criteria

- Every recommendable product has segment, category, palette/style tags, image, link, and variant data.
- Catalog freshness remains inside an agreed service interval.
- Merchant configuration changes storefront behavior without a code deployment.

9. W6 - Commerce attribution and analytics

Priority	Items	Effort	Objective
P1	7	3-5 days	Connect recommendations to checkout outcomes and trustworthy merchant reporting.

Implementation details

- W6.1 Preserve StyledGenie attribution through product view, cart, checkout, and order.
- W6.2 Ingest orders and fulfillment updates via webhook with scheduled reconciliation.
- W6.3 Match assisted line items to recommendation sessions and modes.
- W6.4 Calculate assisted orders, conversion rate, revenue, average order value, and drop-off.
- W6.5 Correct intent and journey event classification inconsistencies.
- W6.6 Add date, mode, category, campaign, and product filters.
- W6.7 Add analytics reconciliation tests against Shopify source totals.

Acceptance criteria

- A test purchase from a recommendation appears as AI-assisted.
- Revenue metrics derive from stored Shopify orders rather than placeholder values.
- Journey totals and classifications can be reconciled and explained.

10. W7 - Platform engineering and AI operations

Priority	Items	Effort	Objective
P1	10	6-9 days	Make the service maintainable, observable, resilient, and cost-controlled.

Implementation details

- W7.1 Split oversized conversation and recommendation services into focused modules.
- W7.2 Move long catalog, Vision, tagging, notification, and analytics work to background jobs.
- W7.3 Add retry, timeout, circuit-breaker, idempotency, and dead-letter behavior.
- W7.4 Define versioned API contracts and consistent error envelopes.
- W7.5 Add structured logs, tracing, latency metrics, health checks, and alerts.
- W7.6 Track AI token cost, Vision cost, model latency, fallback rate, and failure rate.
- W7.7 Create a fixed AI evaluation dataset covering all modes and edge cases.
- W7.8 Add prompt-injection defenses for shopper input, URLs, images, and merchant knowledge.
- W7.9 Add input size limits, image type checks, URL safety controls, and abuse prevention.
- W7.10 Add caching and query/index optimization after measuring actual bottlenecks.

Acceptance criteria

- Slow external systems cannot exhaust web workers or duplicate writes.
- Operators can identify which dependency failed and which shopper journey was affected.
- AI quality and cost regressions are detected before release.

11. W8 - Privacy, commercial readiness, and launch

Priority	Items	Effort	Objective
P2	8	5-8 days	Complete the business, privacy, operational, and release requirements for a public SaaS launch.

Implementation details

- W8.1 Define shopper consent, preference export/deletion, and conversation/image retention.
- W8.2 Add privacy policy, terms, merchant support, and incident-response documentation.
- W8.3 Implement Shopify subscription plans, trial rules, billing state, and usage limits.
- W8.4 Add staging and production environments with managed secrets and migrations.
- W8.5 Add CI/CD, deployment approvals, smoke tests, rollback, backup, and restore drills.
- W8.6 Complete browser, device, theme, localization, currency, and accessibility validation.
- W8.7 Prepare merchant onboarding, empty states, setup progress, and self-service diagnostics.
- W8.8 Run production pilot, resolve launch blockers, and prepare the public release checklist.

Acceptance criteria

- Data collection, retention, export, and deletion are documented and testable.
- Billing state correctly enables, limits, or disables paid functionality.
- A clean production deployment can be rolled back and restored safely.

12. Recommended execution sequence and dependencies

Sequence	Work	Depends on	Release gate
1	W1 Release stabilization	Current working tree	One canonical widget and valid deploy config
2	W2 Core product tests	Stable release artifact	Mandatory behavior regression suite passes
3	W3 Shopify security	Stable API contracts	Authenticated tenant isolation passes
4	W4 Support operations	Shop context and order access	Real tracking, return, and handoff verified
5	W5 Catalog controls	Tenant context and job processing	Fresh, tagged, inventory-safe catalog
6	W6 Analytics	Order sync and attribution metadata	Test purchase reconciles to Shopify
7	W7 Platform operations	Can begin earlier; completes before production	Alerts, retries, evals, and CI active
8	W8 Public launch	All production gates	Billing, privacy, staging, rollback, and pilot complete

Suggested 12-week one-developer roadmap

Weeks	Primary outcome
1	Stabilize sources, assets, config, docs, and smoke checks.
2	Automate mandatory styling/support behavior and resolve discovered regressions.
3-4	Implement Shopify authentication, tenant resolution, protected merchant APIs, and RLS.
5	Connect real support policies, order lookup, returns/exchanges, and email handoff.
6	Automate catalog sync/tagging and validate merchant configuration behavior.
7	Complete attribution, order ingestion, and reconciled analytics.
8-9	Background jobs, error handling, observability, AI evaluation, and security hardening.
10	Billing, privacy operations, onboarding, accessibility, and localization.
11	Staging pilot, performance testing, backup/restore, and release rehearsal.
12	Fix pilot findings, production cutover, monitoring, and controlled launch.

Parallelization note: Small parts can overlap, but W3 must precede any public merchant access, and W4/W6 depend on reliable Shopify shop and order context.

13. Required test and quality matrix

Layer	Required coverage	Minimum release gate
Unit	Routing, profile extraction, gap analysis, ranking, segment filtering, swaps, support intent, policy resolution	Critical rule branches covered with fixed fixtures
API integration	Chat, image, refine, feedback, catalog, merchant writes, support, analytics, authentication	Success, validation, authorization, and dependency-failure cases pass
Contract	Frontend payloads versus Pydantic schemas and stable error envelope	No silent field drift between widget and backend
AI evaluation	Representative fashion, ambiguity, image, support, injection, and failure prompts	Quality thresholds recorded and compared per release
Shopify E2E	Install, embedded dashboard, widget, PDP, cart, checkout, order, return, uninstall	Test store journey completes without manual database edits
Security	Auth bypass, tenant crossover, PII disclosure, rate limit, malicious URL/image, prompt injection	No high-severity open findings
Accessibility	Keyboard, focus, labels, screen reader, contrast, reduced motion, zoom	Core journeys meet agreed accessibility baseline
Compatibility	Mobile/desktop, major browsers, representative Shopify themes	No blocking layout or interaction defects
Performance	Widget load, API latency, image payload, catalog sync, concurrent chat	Budgets defined and measured in staging
Resilience	OpenAI, Vision, Supabase, Shopify, SMTP/Twilio timeouts and partial outage	Actionable fallbacks; no duplicate writes or worker exhaustion

Mandatory shopper acceptance scenarios

- Find My Outfit infers known details, asks only missing critical inputs, asks the decision-mode question, and returns the requested number of outfits.
- Complete My Look analyzes an uploaded garment first, identifies what is present and missing, then returns only compatible missing categories.
- Get Inspired presents one closest hero match followed by compact supporting items.
- Smart Swap changes one item category and preserves the rest of the look.
- Track My Order asks for order number and email, then returns real status without showing policy-first content.
- Returns and exchanges identify the order/item and present actionable refund or exchange choices.
- Every support journey removes styling UI; every recommendation contains a useful Why this works explanation.

14. Risk register and controls

Risk	Likelihood	Impact	Control
Local demo differs from Shopify release	High	High	Canonical source plus automated asset parity check
Unauthenticated merchant mutation	High	Critical	Shopify session verification and route authorization
Cross-merchant data exposure	High for SaaS	Critical	Authenticated tenant context, RLS, isolation tests
AI returns plausible but invalid products	Medium	High	Deterministic validation after generation and inventory checks
Support discloses order data	Medium	Critical	Order plus email match, rate limits, audit logs, negative tests
External API latency blocks requests	High	High	Timeouts, background jobs, retries, circuit breakers
Catalog becomes stale	High	Medium	Webhooks, scheduled reconciliation, freshness alert
AI cost increases unexpectedly	Medium	Medium	Usage budgets, caching, model routing, cost dashboards
Large monolith causes regressions	High	High	Characterization tests before incremental extraction
Production release cannot roll back	Medium	High	Versioned deploys, migration discipline, rollback rehearsal

Do not start a broad frontend or backend rewrite. First lock behavior with tests, establish one release artifact, then extract modules incrementally where it improves reliability and decision clarity.

Decisions required from the product owner

- Choose the next release target: demo-ready, secure single-store production, or public SaaS.
- Confirm whether email handoff is sufficient for the first release or WhatsApp is mandatory.
- Approve customer conversation, image, preference, and order-data retention periods.
- Define paid plans and usage limits only if public SaaS is selected.
- Provide the final returns, exchanges, shipping, privacy, and escalation policies.

15. Final definition of done

Demo-ready

- One canonical widget is visible in both local demo and development Shopify store.
- All four modes complete their happy path on desktop and mobile.
- At least one real order can be tracked and one handoff can be delivered.
- Mandatory mode separation, image-first, hero-first, decision-mode, gap-analysis, and Smart Swap checks pass.

Secure single-store production

- Stable HTTPS hosting, protected merchant APIs, explicit CORS, rate limiting, and audit logs are active.
- Catalog, inventory, orders, support content, attribution, and analytics run against the production store.
- Automated tests, monitoring, alerts, backups, and rollback are operational.
- Customer information is disclosed only after correct verification and retained according to policy.

Public multi-merchant Shopify SaaS

- OAuth/install/uninstall works independently for every merchant.
- Tenant isolation is enforced in application code and database policy.
- Billing, trials, usage enforcement, privacy webhooks, self-service onboarding, and merchant support are live.
- A production pilot passes, public-release documentation is complete, and the operational team can respond to incidents.

Recommended next action: execute W1 Release stabilization, then immediately build W2 characterization tests. These two workstreams protect the value already present in the code and create a safe base for every subsequent feature.

Audit basis

This roadmap was prepared from the repository's AGENTS.md rules, README, detailed functional flows, implementation tickets, production checklist, FastAPI routes and services, storefront and merchant JavaScript, Shopify extension assets/configuration, Supabase schema, runtime endpoint checks, configuration readiness checks, catalog/workspace metrics, and syntax verification. No application source files were modified during the audit; only this report and its temporary build artifacts were created.